UE App Development 623.9[51|52] SS 2026
Institute of Information Technology (ITEC) Primus|Münzer

ASSIGNMENT 04
Due date: 2026/04/23 | 21

Aim: **Mastering User Interaction, Pointer Events, and Canvas Drawing**

1 Touch Control & Coordinate Systems

Goal: *Understand basic pointer input and pixel-perfect distance calculations on a Canvas.*

Technical Hint: Pointer Input

In Compose, use the `pointerInput` Modifier with `detectTapGestures` to react to user interactions. Remember that the coordinates are provided in `Offset` (pixels).

☐ **Setup:** Create a full-screen `Canvas`.

☐ **Initial State:** Draw a red circle at a random position.

☐ **Task: Point-to-Point Interaction**

- When the user taps: Draw a blue circle at the tap position.
- Draw a black line connecting the red and blue circles.
- **Calculation:** Compute the Euclidean distance between both points.
- **UI:** Display the result using `drawText` or a superimposed `Text` composable (e.g., "Distance: 450.2 px").

☐ **Bonus Challenges:**

- **Double Tap:** Teleport the red circle to a new random location.
- **Long Press:** Move the red circle to the current touch position.

2 Gesture Detection & Path Rendering

Goal: *Capture continuous motion and manage custom drawing states.*

- ☐ **Drawing Logic:** Implement a free-hand drawing feature where dragging a finger creates a visible line.
- ☐ **Path Management:** Store coordinates in a `Path` object or a `List<Offset>` and render them using `drawPath`.
- ☐ **Persistence:** Ensure that lifting the finger and touching again starts a *new* separate stroke (do not connect the end of the old stroke to the start of the new one).
- ☐ **Deep Dive - Bonus Challenges:**
  - Add a `Slider` to dynamically adjust the brush size (`strokeWidth`).
  - Implement a "Clear Canvas" feature via a long press gesture.
- ☐ **Conceptual Questions:**
  - Explain the difference between a raw "Touch Event" and a "Gesture".
  - Describe the lifecycle of a touch event: *Down, Move, Up, Cancel*.
  - How does Compose distinguish between a `Tap` and the start of a `Scroll`?

3 Multi-Touch Tracking

Goal: *Handling multiple simultaneous pointers and ID tracking.*

Hint: Pointer IDs

Use `awaitPointerEventScope` to access all active pointers. Each finger is assigned a unique `pointerId` which stays constant until the finger is lifted.

- ☐ **Multi-Finger Support:** Track all active fingers and draw a circle at each touch point.
- ☐ **Visual Identity:** Assign a unique color to each finger based on its `pointerId`.
- ☐ **Feedback:** Display the current number of active fingers as a large overlay text.
- ☐ **Path Tracking:** Each finger should draw its own individual path in its assigned color.
- ☐ **Bonus:** Implement a "Fade Out" effect where paths gradually disappear after a finger is lifted.

4 Advanced Transformations

Goal: *Combining gestures for Pinch-to-Zoom, Rotation, and Panning.*

- ☐ Display an image or a complex shape in the center of the screen.
- ☐ **Transformations:** Use `Modifier.transformable()` to implement:
  - **Pinch-to-Zoom:** Scaling the object.
  - **Two-Finger Rotation:** Rotating the object.
  - **Panning:** Moving the object around the screen.
- ☐ **Reset:** Implement a Double Tap gesture to reset all transformations (Scale = 1.0, Rotation = 0).
- ☐ **Bonus:** Detect a three-finger tap to randomize the object's color or texture.